

Lab-7

Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook.

Theory

Ansible is an open-source automation tool used for: Configuration management, Application deployment and Task automation.

It works in an agentless manner (no software required on client machines), using SSH.

Inventory

The inventory file in Ansible contains the list of hosts or network devices that you want to manage. It is usually located at `/etc/ansible/hosts` by default.

To define hosts in the inventory file, open it in a text editor and add the IP addresses or domain names of your servers. You can organize hosts into groups and assign variables to them.

Example inventory file:

[web]

```
webserver1 ansible_host=192.168.0.10 ansible_user=admin
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

[database]

```
dbserver1 ansible_host=192.168.0.20 ansible_user=admin
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

Explanation:

- Groups like webserver and dbserver help organize machines.
- Ansible uses this file to know where to execute tasks.

Playbooks

Ansible playbooks are YAML files that describe the desired state of your system. Playbooks consist of plays, which are sets of tasks executed on a specific group of hosts.

A simple playbook example:

- name: Install and start Apache

hosts: web

```
become: true

tasks:

- name: Install Apache package

  apt:

    name: apache2

    state: present


- name: Start Apache service

  service:

    name: apache2

    state: started

    enabled: true
```

Save the above YAML code to a file named webserver.yml.

Explanation:

- hosts: target machines
- tasks: list of actions
- become: runs tasks as root (sudo)

Modules

Modules are reusable units that perform specific tasks.

Common Modules:

- yum / apt → install packages
- service → manage services
- copy → copy files
- file → manage file properties

Example:

```
- name: Start Apache

  service:

    name: httpd

    state: started
```

Automating Server Configurations with Playbooks

Ansible playbooks automate repetitive server tasks such as:

- Installing software
- Configuring services
- Managing users
- Updating systems

Workflow:

1. Define inventory (servers)
2. Write playbook (tasks)
3. Run playbook
4. Ansible connects via SSH and executes tasks

Advantages:

- Saves time
- Reduces human error
- Ensures consistency across servers

Ansible Control Node vs Managed Nodes

1. Ansible Control Node (Master Node)

The Control Node is the system where Ansible is installed and from which all automation tasks are executed.

Key Functions:

- Runs Ansible commands and playbooks
- Stores:
 - Inventory file
 - Playbooks (YAML files)
 - Configuration files
- Connects to managed nodes using SSH

2. Managed Nodes (Target Nodes / Hosts)

Managed Nodes are the remote systems that Ansible controls.

Key Functions:

- Execute tasks sent by the control node

- Perform operations like:
 - Installing software
 - Starting services
 - Updating configurations

Procedures

Step 1: Update your system

Open Terminal and run:

```
sudo apt update && sudo apt upgrade -y
```

Step 2: Install Ansible

```
sudo apt install ansible -y
```

Check:

```
ansible --version
```

Step 3: Install SSH (needed for Ansible)

```
sudo apt install openssh-client -y
```

Step 4: Create your project folder

```
mkdir ansible_project
```

```
cd ansible_project
```

Step 5: Create Inventory file

```
nano inventory
```

Paste:

```
[local]
```

```
localhost ansible_connection=local
```

Save and exit.

Step 6: Test Ansible

```
ansible -i inventory all -m ping
```

Output should be:

```
pong
```

Step 7: Create Playbook

```
nano playbook.yml
```

Paste:

```
- name: Install nginx
  hosts: local
  become: yes
  tasks:
    - name: Install nginx
      apt:
        name: nginx
        state: present
    - name: Start nginx
      service:
        name: nginx
        state: started
```

Step 8: Run Playbook

```
ansible-playbook -i inventory playbook.yml
```

Step 9: Verify

```
curl http://localhost
```

If you see HTML → SUCCESS 🎉